

Ros Day04 hw1 보고서

작성자: Doyeon Park

작성일: 2026.09.14

목차

1. 과제 개요
2. Lifecycle 의 개념
3. Lifecycle Callback
4. Lifecycle 을 적용해야 하는 이유
5. 적용 패키지 구성
6. QoS 의 개념
7. 각 Node 의 QoS 설정
8. QoS 호환성
9. Lifecycle 과 QoS 의 결합
10. 결론

1. 과제 개요

일반적인 ROS2 Node 는 실행되는 순간부터 바로 자신의 기능을 수행한다. 간단한 프로그램에서는 이러한 방식으로 문제가 없지만, 실제 로봇 시스템에서는 카메라, LiDAR, 모터와 같은 여러 장치가 동시에 사용되기 때문에 각 장치의 준비 상태와 실행 순서를 관리할 필요가 있다.

예를 들어 LiDAR 센서가 아직 초기화되지 않았는데 제어 노드가 먼저 실행된다면 정상적인 센서 데이터를 받지 못한 상태에서 로봇의 이동 명령이 생성될 수 있다. 또한 프로그램을 종료하거나 다시 시작하는 과정에서도 사용하고 있던 장치나 통신 자원을 적절하게 정리해야 한다.

ROS2 에서는 이러한 문제를 해결하기 위해 Lifecycle Node 를 사용할 수 있다. Lifecycle Node 는 노드를 단순히 실행하고 종료하는 방식이 아니라, 노드의 상태를 여러 단계로 구분하여 관리한다. 자료에서는 Lifecycle Node 가 정해진 State Machine 과 Transition 을 제공하며 초기화가 완료된 이후에만 Active 상태로 전환하고, deactivate, cleanup, shutdown 등을 통해 동작을 관리할 수 있다.

2. Lifecycle 의 개념

Lifecycle Node 에는 크게 Unconfigured, Inactive, Active, Finalized 의 네 가지 주요 상태가 존재한다.

Unconfigured 는 Node 가 처음 생성된 상태로 아직 장치나 Publisher, Subscriber 와 같은 자원이 준비되지 않은 상태이다. configure Transition 을 수행하면 필요한 초기화 과정이 진행되고, 초기화에 성공하면 Inactive 상태로 이동한다.

Inactive 상태에서는 필요한 설정과 자원이 준비되어 있지만 실제 데이터 처리나 Publish 와 같은 주요 기능은 수행하지 않는다. 따라서 장치 준비는 완료했지만 아직 로봇이 동작해서는 안 되는 상태라고 볼 수 있다.

이후 activate 를 수행하면 Active 상태가 된다. Active 상태에서는 실제 데이터 처리와 Publish, Subscribe 등의 기능이 수행된다. 다시 deactivate 를 수행하면 Inactive 상태로 돌아갈 수 있다.

마지막으로 Finalized 는 노드가 종료되기 직전의 상태이다. Lifecycle 의 주요 상태와 상태 전환 과정은 Unconfigured → Inactive → Active 로 볼 수 있다.

이러한 상태 관리 방식을 사용하면 프로그램을 단순히 실행하고 종료하는 것이 아니라 각 노드가 현재 어떤 준비 상태에 있는지 명확하게 확인할 수 있다.

3. Lifecycle Callback

Lifecycle 에서는 각 상태가 변경될 때 Callback 함수를 실행하여 해당 단계에서 필요한 작업을 할 수 있다.

on_configure()에서는 Publisher, Subscriber, Parameter 설정이나 장치 초기화처럼 프로그램이 실제로 동작하기 전에 필요한 자원을 준비한다. 센서를 사용하는 프로그램이라면 센서 연결이나 통신 설정 등을 이 단계에서 수행할 수 있다.

on_activate()에서는 Timer 나 Publisher 등을 실제로 동작시키면서 노드의 주요 기능을 시작한다. 따라서 이동 로봇에서는 센서 데이터 송신이나 제어 명령 송신을 이 시점부터 시작할 수 있다.

반대로 on_deactivate()에서는 Active 상태에서 수행되고 있던 기능을 중지한다. 로봇을 예로 들면 센서 데이터 처리를 중지하거나 모터 제어 명령을 중단하는 작업을 수행할 수 있다.

on_cleanup()은 configure 과정에서 생성했던 자원을 제거하고 초기 상태에 가까운 상태로 되돌리는 역할을 한다. 마지막으로 on_shutdown()에서는 Node 가 종료되기 전에 필요한 자원을 최종적으로 정리한다. 이러한 Callback 들의 역할도 각각 준비, 시작, 중지, 정리 과정으로되어 있다.

따라서 Lifecycle 을 사용하면 하나의 함수 안에서 모든 초기화와 실행, 종료 과정을 처리하는 것보다 프로그램의 동작 단계를 명확하게 분리할 수 있다는 장점이 있다.

4. Lifecycle 을 적용해야 하는 이유

이동 로봇 시스템에서는 여러 노드가 동시에 실행된다. 예를 들어 LiDAR 센서로 주변 환경을 측정하는 노드, 센서 데이터를 이용하여 이동 명령을 계산하는 제어 노드, 그리고 현재 로봇의 상태를 다른 노드에 알려주는 상태 관리 노드 등이 사용될 수 있다.

이러한 시스템에서 모든 노드가 실행 직후 바로 동작한다면 초기화 순서에 따라 문제가 발생할 가능성이 있다. 제어 노드가 먼저 실행되고 LiDAR 센서가 정상적으로 준비되지 않았다면 잘못된 데이터 또는 데이터가 없는 상태에서 제어 연산을 수행할 수 있기 때문이다.

Lifecycle 을 사용하면 먼저 각 노드를 configure 하여 필요한 장치와 ROS 통신 자원을 준비하고, 모든 준비가 정상적으로 완료되었을 때만 activate 하여 실제 동작을 시작할 수 있다.

또한 로봇 동작 중 문제가 발생했을 경우 프로그램 자체를 완전히 종료하지 않고 deactivate 하여 주요 기능만 정지시킬 수 있다. 이후 문제를 해결한 뒤 다시 activate 하면 시스템을 다시 동작시킬 수 있다.

이러한 특성 때문에 카메라, LiDAR, 모터처럼 준비 순서가 중요한 장치를 안전하고 예측 가능하게 시작하거나 중지할 때 Lifecycle 이 적합하다. Lifecycle 이 필요한 대표적인 이유로 이러한 장치들의 준비 순서와 안전한 시작·중지·재설정등이 있다.

5. 적용 패키지 구성

Lifecycle 을 적용하기 적합한 예시로 robot_lifecycle_pkg 라는 이동 로봇 패키지를 가정해보자.

패키지는 lidar_node, controller_node, robot_status_node 의 세 개 Node 로 구성하자.

lidar_node 는 LiDAR 센서로부터 거리 정보를 읽어 ROS Topic 으로 Publish 한다. controller_node 는 LiDAR 데이터를 Subscribe 하여 장애물과의 거리를 확인하고 로봇의 속도 또는 이동 명령을 생성한다. robot_status_node 는 현재 로봇의 동작 상태를 다른 노드에게 전달한다.

세 노드는 서로 다른 종류의 데이터를 사용하기 때문에 모든 Topic 에 동일한 QoS 를 사용하는 것보다 각 데이터 특성에 맞는 QoS 를 설정하는 것이 적절하다고 볼 수 있다.

6. QoS 의 개념

QoS 는 Quality of Service 의 약자로 Publisher 와 Subscriber 사이의 통신 방식을 결정하는 여러 Policy 의 집합이다.

ROS2 에서는 Topic 이름과 Message Type 이 동일하더라도 QoS 설정이 서로 호환되지 않으면 정상적으로 메시지를 주고받을 수 없다. 자료에서는 QoS 의 대표적인 설정으로 History, Depth, Reliability, Durability 를 설명하고 있다.

History 는 메시지를 어떤 방식으로 저장할 것인지를 설정한다. Keep Last 는 최근 메시지를 정해진 개수만큼 보관하며, Keep All 은 Middleware 의 Resource Limit 내에서 가능한 메시지를 모두 보관한다.

Depth 는 Keep Last 를 사용했을 때 몇 개의 메시지를 Queue 에 저장할 것인지를 의미한다. 즉, 메시지를 몇 번 전송한다는 의미가 아니라 최근 메시지를 몇 개까지 유지할 것인지를 의미한다.

Reliability 에는 Reliable 과 Best Effort 가 있다. Reliable 은 메시지가 손실될 경우 재전송을 시도하여 전달 신뢰성을 높인다. 반대로 Best Effort 는 재전송을 보장하지 않지만 최신 데이터를 빠르게 전달하는 데 유리하다. 따라서 카메라나 LiDAR 처럼 높은 주기로 계속 새로운 데이터가 들어오는 센서에는 Best Effort 가 유리하며, 중요한 상태 정보나 명령에는 Reliable 이 유리하다.

Durability 에는 Volatile 과 Transient Local 이 있다. Volatile 은 Subscriber 가 연결된 이후에 발생한 메시지만 전달하지만 Transient Local 은 늦게 연결된 Subscriber 도 Publisher 가 보관하고 있는 이전 메시지를 받을 수 있게 한다.

7. 각 Node 의 QoS 설정

각 노드에서 사용하는 데이터의 특성을 고려하여 다음과 같이 QoS 를 설정하였다

노드	QoS 설정	선택 이유
lidar_node	Keep Last, Depth 5, Best Effort, Volatile	LiDAR 는 오래된 데이터보다 최신 데이터 전달이 중요하므로 Best Effort 사용
controller_node	Keep Last, Depth 1, Reliable, Volatile	제어 명령 손실을 줄이고 최신 명령만 유지하기 위해 설정
robot_status_node	Keep Last, Depth 1, Reliable, Transient Local	늦게 연결된 노드도 현재 상태를 받을 수 있도록 설정

LiDAR 데이터는 높은 주기로 계속 갱신되므로 오래된 데이터의 완전한 전달보다 최신 데이터 전달이 중요하다고 판단하여 Best Effort 를 적용하였다. 제어 명령은 손실될 경우 로봇 동작에 영향을 줄 수 있으므로 Reliable 을 적용하였다. 상태 정보는 새로 연결된 Subscriber 도 현재 상태를 확인할 필요가 있으므로 Transient Local 을 사용하였다.

8. QoS 호환성

QoS 를 설정할 때는 각각의 값을 단순히 선택하는 것뿐만 아니라 Publisher 와 Subscriber 사이의 호환성도 고려해야 한다.

ROS2 에서는 Publisher 가 제공하는 QoS 를 Subscriber 가 요구하는 QoS 가 만족할 수 있어야 통신이 가능하다. 예를 들어 Publisher 가 Best Effort 로 데이터를 보내는데 Subscriber 가 Reliable 통신을 요구한다면 Subscriber 의 요구를 Publisher 가 만족하지 못하므로 통신할 수 없다.

반대로 Reliable Publisher 와 Best Effort Subscriber 의 조합은 가능하다.

Durability 에서도 Volatile Publisher 와 Transient Local Subscriber 의 조합은 호환되지 않는다. 따라서 하나의 Topic 을 사용하는 Publisher 와 Subscriber 를 설계할 때는 양쪽 QoS 설정을 같이 확인해야 한다.

9. Lifecycle 과 QoS 의 결합

Lifecycle 과 QoS 를 함께 사용하면 Node 의 실행 상태와 통신 특성을 동시에 관리할 수 있다.

먼저 Node 가 생성되면 Unconfigured 상태가 된다. 이후 `on_configure()`에서 각 Node 에 필요한 QoS Profile 을 생성하고 Publisher 와 Subscriber 등의 통신 자원을 준비한다.

설정이 완료되면 Node 는 Inactive 상태가 된다. 이 상태에서는 통신을 위한 준비는 완료되어 있지만 실제 로봇 기능은 수행하지 않는다.

activate Transition 이 실행되면 Active 상태로 변경되고 실제 Publish, Subscribe 와 데이터 처리를 시작한다. 이후 로봇을 일시적으로 정지해야 하는 경우 deactivate 를 사용하여 Inactive 상태로 돌아가고, 더 이상 사용하지 않는 경우 cleanup 또는 shutdown 으로 자원을 정리한다.

즉, QoS 는 어떤 방식으로 통신할 것인가를 결정하고 Lifecycle 은 언제 통신과 실제 기능을 수행할 것인가를 결정한다고 볼 수 있다.

10. 결론

이번 보고서를 작성하며 Lifecycle 은 단순히 Node 의 실행과 종료를 나누는 기능이 아니라 로봇 시스템의 초기화, 대기, 동작, 정리 과정을 명확하게 관리하기 위한 기능이라는 것을 알 수 있었다.

특히 LiDAR, 모터와 같이 준비 순서가 중요한 장치를 사용하는 이동 로봇에서는 모든 장치가 준비되기 전에 로봇이 동작하면 예기치 않은 문제가 발생할 수 있다. Lifecycle 을 사용하면 이러한 문제를 줄이고 시스템의 상태를 일정한 방식으로 관리할 수 있다는 것을 알게 되었다.

결과적으로 실제 ROS2 로봇 시스템을 설계할 때는 Lifecycle 을 통해 Node 의 실행 상태를 관리하고, QoS 를 통해 각 Topic 의 통신 특성을 관리하는 방식이 안정적인 시스템 구성에 도움이 될 수 있다 생각한다.